

Discrete Wave-Constrained Computation and Classical Complexity: Turing Equivalence for P and NP

Richard J. Reyes

November 26, 2025

Original Release: November 26, 2025

Abstract

Wave Confinement Theory (WCT) motivates a discrete model of computation in which information is stored and processed by local interactions between finitely many metastable states arranged on a lattice. In this paper we isolate the resulting *wave-constrained computation* (WCC) model as a finite-state, finite-range local-update machine and compare its complexity classes to the classical ones.

First, we define discrete WCC machines under the standard “rails” (locality, finite precision, bounded propagation) and construct an explicit simulation of any polynomial-time deterministic Turing machine by a polynomial-time WCC machine with only constant-factor overhead in time and polynomial overhead in space. Conversely, we show that any polynomial-time WCC machine can be simulated by a deterministic Turing machine with polynomial overhead. This yields $P_{\text{WCC}} = P_{\text{Turing}}$.

We then lift this equivalence from decision procedures to verifier-based nondeterministic computation. We define NP_{WCC} via polynomial-time WCC verifiers with polynomially bounded witnesses, and prove that $NP_{\text{WCC}} = NP_{\text{Turing}}$. Taken together, these results show that, at the level of their discrete abstraction, WCT-compatible computation is neither sub-Turing nor super-Turing in power: it realizes exactly the classical P and NP classes.

0.1 Resource Model Summary

For convenience, we summarize the resource model used throughout the definition of discrete WCC machines. Let $x \in \Sigma^n$ be an input of length n .

1. **Space (Finite Lattice).** Computation takes place on a finite lattice $\Lambda_n \subset \mathbb{Z}^d$ (with fixed d) satisfying

$$|\Lambda_n| \leq p(n)$$

for some polynomial p . The cardinality $|\Lambda_n|$ represents the space cost of the machine.

2. **Precision (Finite State).** The state set S is finite. Each cell holds an element of S , representing $\ell = \log_2 |S|$ bits of information. No operation increases ℓ ; precision remains fixed and finite throughout computation.
3. **Locality (Neighborhood Radius).** A fixed interaction radius $r \in \mathbb{N}$ defines neighborhoods

$$N_r(i) = \{j \in \Lambda_n : \|i - j\|_\infty \leq r\}.$$

Each update applies a local rule $U : S^{N_r} \rightarrow S$.

4. **Determinism and Time.** Computation proceeds in synchronous, deterministic rounds via U . For a WCC machine W , the time cost on inputs of length n is

$$T_W(n) = \max_{x \in \Sigma^n} \min\{t : \text{dec}(C_t) \in \{0, 1\}\}.$$

Given C_0 and U , the entire trajectory $(C_t)_{t \geq 0}$ is uniquely determined (deterministic replay).

5. **Total Configuration Space.** Since $|\Lambda_n|$ and $|S|$ are finite, the global configuration space S^{Λ_n} is finite. This precludes hypercomputation via infinite-dimensional state spaces.

This model supports direct comparison with deterministic Turing machines in uniform polynomial time.

Lemma 0.1 (Precision Stability). *Let $W = (\{\Lambda_n\}, S, r, U, \text{enc}, \text{dec})$ be a discrete WCC machine. If $C_t : \Lambda_n \rightarrow S$ is representable using $\ell = \log_2 |S|$ bits per cell, then so is C_{t+1} obtained by the local update rule U .*

Proof. For each $i \in \Lambda_n$,

$$C_{t+1}(i) = U(C_t|_{N_r(i)}).$$

Since $U : S^{N_r} \rightarrow S$ is a total function whose image lies in the same finite state set S , its outputs require no more than $\log_2 |S|$ bits. No arithmetic involving unbounded precision is performed. Thus, the precision requirement remains constant, preventing accumulation of infinite information during evolution. $\square$

Proposition 0.2 (Local Implementability of TM Transitions). *Let M be a deterministic single-tape Turing machine with transition function*

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}.$$

Under the two-phase head/symbol encoding of Section 1, each transition of M is realizable by a WCC local update rule with radius $r = 1$.

Proof. In the simulation, each tape cell is encoded as a WCC state $(\gamma, q^\#, p) \in S$, where $\gamma \in \Gamma$, $q^\# \in Q \cup \{\perp, (q, L), (q, R)\}$ encodes the head state or a directional mark, and $p \in \{0, 1\}$ is a phase bit.

To realize a TM transition, the rule U need only inspect the cell currently holding the head and its immediate neighbors (to which the head may move in the next phase). These cells lie inside the radius-1 neighborhood $\{i - 1, i, i + 1\}$.

Given (q, γ) at the head location, U rewrites the symbol, enters the next state, and places a directional mark for the subsequent phase. Movement of the head by one cell is therefore computed using only local information. Hence, all TM transitions are locally implementable in the WCC model. $\square$

Proposition 0.3 (Deterministic Evolution and Replay). *Let W be a discrete WCC machine. Then the global evolution*

$$C_{t+1}(i) = U(C_t|_{N_r(i)})$$

is deterministic. Consequently, given C_0 and U , the entire trajectory $(C_t)_{t \geq 0}$ is uniquely determined and can be reconstructed (replayed) by any verifier.

Proof. Since U is a function from the finite domain S^{N_r} to the finite set S , each cell update is uniquely determined by the configuration of its neighborhood. Thus C_{t+1} is uniquely determined by C_t .

By induction, the full sequence of configurations is uniquely determined by C_0 . A verifier with access to C_0 and U can therefore reproduce the entire evolution without ambiguity, establishing deterministic replay. $\square$

1 Discrete WCC Machines and Turing Equivalence

In this section we isolate the discrete core of wave-constrained computation (WCC) as a finite-state local-update machine model and prove that its polynomial-time class coincides with the classical deterministic polynomial-time class. This makes precise the identification

$$P_{\text{WCC}} = P_{\text{Turing}}$$

used elsewhere in the paper.

We first define the discrete WCC machine under the locality and precision rails, then construct an explicit simulation of any polynomial-time Turing machine by a polynomial-time WCC machine with only constant-factor overhead in time. We then recall the converse simulation (WCC by Turing machines) and obtain full equivalence.

1.1 Discrete WCC machine model

Let Σ be a finite input alphabet and let $n = |x|$ denote input length.

Definition 1.1 (Discrete WCC machine). *A discrete WCC machine is a tuple*

$$W = (\{\Lambda_n\}_{n \geq 1}, S, r, U, \text{enc}, \text{dec})$$

with the following components.

- *For each input length n , a finite index set*

$$\Lambda_n \subset \mathbb{Z}^d$$

representing the lattice of computational cells. We require a polynomial bound

$$|\Lambda_n| \leq p(n)$$

for some fixed polynomial p (finite-precision rail (P)).

- *A finite state set S of cell symbols.*
- *A fixed interaction radius $r \in \mathbb{N}$ and a neighbourhood shape*

$$N_r(i) := \{j \in \mathbb{Z}^d : \|j - i\|_\infty \leq r\}$$

for each $i \in \Lambda_n$.

- A local update rule

$$U : S^{N_r} \rightarrow S$$

which induces a global synchronous update

$$C_{t+1}(i) = U(C_t|_{N_r(i)}), \quad i \in \Lambda_n$$

on configurations $C_t : \Lambda_n \rightarrow S$. This enforces the locality and bounded-propagation rails (L, V) .

- An encoding map

$$\text{enc} : \Sigma^* \rightarrow \bigsqcup_{n \geq 1} S^{\Lambda_n}$$

which assigns to each input $x \in \Sigma^n$ an initial configuration $C_0 = \text{enc}(x) \in S^{\Lambda_n}$.

- A decoding predicate

$$\text{dec} : S^{\Lambda_n} \rightarrow \{0, 1\}$$

which decides acceptance based on a configuration.

Given an input $x \in \Sigma^n$, the machine W produces a trajectory

$$C_0 = \text{enc}(x), \quad C_{t+1}(i) = U(C_t|_{N_r(i)}),$$

and we say that W accepts x at time t if $\text{dec}(C_t) = 1$.

Definition 1.2 (Polynomial-time WCC). Let $T_W(n)$ be the worst-case number of update steps until $\text{dec}(C_t)$ stabilizes on an input $x \in \Sigma^n$, and assume that one update step (a single global pass of U over Λ_n) can be effected with cost polynomial in $|\Lambda_n|$.

We say that W decides a language $L \subseteq \Sigma^*$ in polynomial time if there exists $k \geq 1$ such that for all inputs $x \in \Sigma^n$,

$$T_W(n) \leq n^k$$

and W accepts x if and only if $x \in L$.

The corresponding complexity class P_{WCC} consists of all languages L for which there exists a polynomial-time WCC machine in the sense above.

The remaining WCC rails (coercivity, entropy pruning, renewability) impose additional structure on how such a discrete machine can arise from an underlying wave-evolution model. In this section we restrict attention to the discrete interface; the physical realization is addressed separately (Remark 1.19).

1.2 Wave–Symbol Isomorphism

In the WCT/WCC setting, physical computation is realized as the discrete evolution of a curvature-regulated wavefield Ψ_t under locality and finite precision rails. This subsection formalizes the correspondence between such ψ -evolutions and the symbolic WCC update rule

$$U : S^{N_r} \rightarrow S.$$

Definition 1.3 (Curvature-regulated ψ -evolution). *Fix $d \geq 1$. For each input length n , let $\Lambda_n \subset \mathbb{Z}^d$ be the finite lattice of computational cells and let*

$$\Psi_t : \Lambda_n \rightarrow \mathbb{C}$$

be the discretized ψ -field at step $t \in \mathbb{N}$. A curvature-regulated ψ -evolution with interaction radius $r \in \mathbb{N}$ is a family of maps

$$F : \mathbb{C}^{N_r} \rightarrow \mathbb{C}$$

such that the update rule

$$\Psi_{t+1}(i) = F(\Psi_t|_{N_r(i)}), \quad i \in \Lambda_n, \quad (1)$$

is well-defined and satisfies the WCT rails:

- (P) **Finite precision:** *each $\Psi_t(i)$ is representable with at most b bits for some fixed b independent of t and n ;*
- (L) **Locality:** *F depends only on the values in the radius- r neighbourhood $N_r(i)$;*
- (V) **Bounded propagation:** *information cannot travel faster than r cells per step, as enforced by (1).*

Definition 1.4 (Quantization pair). *Let S be a finite state set. A quantization pair for ψ -evolutions is a pair of maps*

$$Q : \mathbb{C} \rightarrow S, \quad \tilde{Q} : S \rightarrow \mathbb{C},$$

such that Q is surjective, $\tilde{Q}$ selects one representative per symbol, and the induced symbolic configuration

$$C_t := Q \circ \Psi_t : \Lambda_n \rightarrow S$$

retains at most b bits per cell (for some b) and respects the WCT precision rail.

Given $(F, Q, \tilde{Q})$, we construct a local symbolic update rule $U : S^{N_r} \rightarrow S$ by

$$U(c|_{N_r(i)}) := Q\left(F(\tilde{Q}(c)|_{N_r(i)})\right), \quad c \in S^{\Lambda_n}, \quad i \in \Lambda_n. \quad (2)$$

Theorem 1.5 (Wave-Symbol Isomorphism). *Let $(\Psi_t)_{t \geq 0}$ be a curvature-regulated ψ -evolution on Λ_n with local update (1) and rails (P, L, V) . Let $(Q, \tilde{Q})$ be a quantization pair with finite alphabet S , and define U by (2). Then:*

1. *The symbolic configurations*

$$C_t := Q \circ \Psi_t : \Lambda_n \rightarrow S$$

satisfy the WCC local-update rule

$$C_{t+1}(i) = U(C_t|_{N_r(i)}), \quad i \in \Lambda_n. \quad (3)$$

2. *Conversely, given any symbolic WCC trajectory $(C_t)_{t \geq 0}$ obeying (3), the reconstructed wavefield*

$$\Psi_t := \tilde{Q} \circ C_t$$

evolves according to a curvature-regulated local update of the form (1) with

$$F(z|_{N_r(i)}) := \tilde{Q}\left(U(Q(z)|_{N_r(i)})\right).$$

In particular, under rails (P, L, V) , curvature-regulated ψ -evolutions and discrete WCC machines are related by a local, finite-state isomorphism:

$$\psi\text{-evolution} \longleftrightarrow \text{symbolic WCC update}.$$

Proof. (1) For each $i \in \Lambda_n$, we have

$$\begin{aligned} C_{t+1}(i) &= Q(\Psi_{t+1}(i)) = Q\left(F(\Psi_t|_{N_r(i)})\right) \\ &= Q\left(F(\tilde{Q}(Q(\Psi_t))|_{N_r(i)})\right) = Q\left(F(\tilde{Q}(C_t)|_{N_r(i)})\right) \\ &= U(C_t|_{N_r(i)}), \end{aligned}$$

by the definition (2). This yields (3).

(2) Given (C_t) with $C_{t+1}(i) = U(C_t|_{N_r(i)})$ and $\Psi_t = \tilde{Q}(C_t)$, define F as above. Then

$$\begin{aligned}\Psi_{t+1}(i) &= \tilde{Q}(C_{t+1}(i)) = \tilde{Q}(U(C_t|_{N_r(i)})) \\ &= \tilde{Q}\left(U(Q(\Psi_t)|_{N_r(i)})\right) = F(\Psi_t|_{N_r(i)}),\end{aligned}$$

which has precisely the local form (1). Finiteness of S and bounded radius r ensure that F respects the WCT rails (P,L,V). $\square$

Theorem 1.5 formalizes the slogan “*computation = ψ -evolution; logic = curvature stencil*”: under WCT rails, every physically admissible ψ -trajectory induces a finite-state WCC computation, and every WCC trajectory admits a ψ -realization.

1.3 Gandy Correspondence and Physical Cellular Automata

Gandy’s physical Church–Turing thesis characterizes a broad class of “reasonable” physical computation models by four structural axioms: bounded information density, spatial homogeneity, bounded propagation speed, and local causality. In this subsection we show that discrete WCC machines satisfy these axioms and therefore instantiate a physical cellular automaton in Gandy’s sense.

Definition 1.6 (Gandy-style physical axioms). *A discrete dynamical system on configurations $C : \Lambda \rightarrow S$ satisfies Gandy’s physical constraints if:*

1. **Bounded information density:** *S is finite, so each site carries at most $\log_2 |S|$ bits of information.*
2. **Spatial homogeneity:** *There exists a fixed local rule $U : S^{N_r} \rightarrow S$ independent of the site $i \in \Lambda$ and of the configuration size.*
3. **Bounded propagation:** *There is a fixed radius $r \in \mathbb{N}$ such that the update at site i depends only on the values in the neighbourhood $N_r(i)$.*
4. **Local causality:** *The global update $C \mapsto C'$ is defined pointwise by*

$$C'(i) = U(C|_{N_r(i)}), \quad i \in \Lambda,$$

with no dependence on nonlocal aggregates.

Proposition 1.7 (WCC satisfies Gandy axioms). *Let*

$$W = (\{\Lambda_n\}_{n \geq 1}, S, r, U, \text{enc}, \text{dec})$$

be a discrete WCC machine as in Definition ???. Then for each n the induced evolution on configurations $C_t : \Lambda_n \rightarrow S$ satisfies all four Gandy-style physical axioms.

Proof. Fix $n \geq 1$.

(1) *Bounded information density.* By definition of a WCC machine, the state set S is finite. Thus each cell carries at most $\log_2 |S|$ bits of information.

(2) *Spatial homogeneity.* The local update rule $U : S^{N_r} \rightarrow S$ is fixed once and for all in the specification of W , and does not depend on $i \in \Lambda_n$ or on n .

(3) *Bounded propagation.* The neighbourhood shape $N_r(i)$ is defined with a fixed radius $r \in \mathbb{N}$. Therefore the update at site i cannot depend on configuration values outside $N_r(i)$, and information cannot propagate faster than r cells per global step.

(4) *Local causality.* By definition of the WCC evolution,

$$C_{t+1}(i) = U(C_t|_{N_r(i)}),$$

so the future value at site i depends only on the local neighbourhood $N_r(i)$ at time t and not on any global property of C_t .

Thus all four axioms hold for the evolution on Λ_n , for each n . $\square$

Corollary 1.8 (Gandy–WCC correspondence). *Every discrete WCC machine induces, for each input length n , a physical cellular automaton in the sense of Gandy. Combined with the TM–WCC simulation results of Section ??, this places WCC firmly within the class of physically realizable, Turing-equivalent models covered by Gandy’s theorem.*

Proposition 1.7 and Corollary 1.8 make explicit that WCC is not only a discrete abstraction compatible with Turing machines, but also a physically grounded local-update model satisfying the standard axioms for realizable computation.

1.4 Deterministic Turing machines

We recall the standard deterministic Turing machine model and its polynomial-time class.

Definition 1.9 (Deterministic Turing machine). *A deterministic Turing machine is a tuple*

$$M = (Q, \Gamma, \Sigma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$$

where:

- Q is a finite set of states.
- Γ is a finite tape alphabet containing a distinguished blank symbol $\sqcup$.
- $\Sigma \subseteq \Gamma \setminus \{\sqcup\}$ is the input alphabet.
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function.
- $q_0 \in Q$ is the start state.
- $q_{\text{acc}}, q_{\text{rej}} \in Q$ are halting accept/reject states.

A configuration of M is a triple (γ, h, q) consisting of a bi-infinite tape $\gamma : \mathbb{Z} \rightarrow \Gamma$ with all but finitely many entries equal to $\sqcup$, a head position $h \in \mathbb{Z}$, and a current state $q \in Q$. One transition step applies $\delta(q, \gamma(h))$ and updates (γ, h, q) accordingly.

Definition 1.10 (Polynomial-time Turing computation). *Let $T_M(n)$ denote the worst-case number of transition steps that M uses on inputs of length n . A language $L \subseteq \Sigma^*$ is in P_{Turing} if there exists a machine M and constants $c, k \geq 1$ such that*

$$T_M(n) \leq cn^k$$

for all n , and M accepts x if and only if $x \in L$.

1.5 Simulation of polynomial-time Turing machines by WCC

We now construct, for each polynomial-time Turing machine M , a discrete WCC machine W_M that simulates M with only constant-factor overhead in time. The construction proceeds via a one-dimensional radius-1 local update rule that encodes the tape and head in per-cell symbols and uses a two-phase schedule to implement the Turing transition function.

1.5.1 Configuration encoding

Fix a Turing machine M as in Definition 1.9. We first define the local alphabet used by the WCC machine.

Definition 1.11 (Local tape state and phase). *Let*

$$S_{\text{tape}} := \Gamma \times (Q \cup \{\perp\})$$

where the symbol $\perp$ denotes “no head present”. A local tape state $(\gamma, q) \in S_{\text{tape}}$ thus records the tape symbol $\gamma \in \Gamma$ and, if $q \in Q$, the presence of the head with state q at the corresponding cell.

We extend the state space with a phase bit and a directional mark. Let

$$Q' := Q \cup \{(q, L) : q \in Q\} \cup \{(q, R) : q \in Q\}$$

and define

$$S := \Gamma \times (Q' \cup \{\perp\}) \times \{0, 1\}.$$

An element $s = (\gamma, q^\#, p) \in S$ consists of:

- a symbol $\gamma \in \Gamma$,
- either $q^\# = \perp$ (no head), or $q^\# \in Q$ (head present with state $q^\#$), or $q^\# = (q, d)$ with $q \in Q$ and $d \in \{L, R\}$ indicating that in the next phase the head should move one cell in direction d ,
- a phase bit $p \in \{0, 1\}$. Phase $p = 0$ corresponds to “before Turing transition” and phase $p = 1$ to “after symbol rewrite, before head movement”.

Given a Turing configuration (γ, h, q) at time t , we encode it as a configuration $C_t : \mathbb{Z} \rightarrow S$ by

$$C_t(i) := \begin{cases} (\gamma(i), q, 0), & \text{if } i = h, \\ (\gamma(i), \perp, 0), & \text{if } i \neq h. \end{cases}$$

In the WCC machine we restrict to a finite window

$$\Lambda_n := \{-L(n), \dots, L(n)\} \subset \mathbb{Z}$$

with $L(n)$ chosen as a polynomial upper bound on the space usage of M on inputs of length n , for example

$$L(n) := T_M(n) + n.$$

1.5.2 Local update rule

We now define the local rule $U : S^{\{-1,0,1\}} \rightarrow S$, which acts with radius $r = 1$ in dimension $d = 1$. For each site $i \in \Lambda_n$, the neighbourhood is

$$N_1(i) = \{i - 1, i, i + 1\}$$

(with the understanding that we only care about neighbourhoods fully contained in Λ_n ; since the head never reaches the boundary within $T_M(n)$ steps, the behaviour of U at the outermost sites can be defined arbitrarily).

The action of U depends on the phase bit of the centre cell.

Phase 0: symbol rewrite and direction tagging. In phase $p = 0$, at most one cell carries a head state in Q , all other cells have $q^\# = \perp$. For a cell at site i with local state $s_i = (\gamma_i, q_i^\#, 0)$, we define the phase-0 component U_{phase0} as:

- If $q_i^\# \in Q$ (the head is present at i with state $q_i^\#$), we consult the Turing transition function

$$\delta(q_i^\#, \gamma_i) = (q', \gamma', d), \quad d \in \{L, R\},$$

and set

$$U_{\text{phase0}}(s_i) := (\gamma', (q', d), 1),$$

that is, we overwrite the symbol with γ' , store a directional mark (q', d) , and flip the phase bit to 1.

- If $q_i^\# = \perp$ (no head), then the cell does not change its symbol or head component and only flips the phase:

$$U_{\text{phase0}}(s_i) := (\gamma_i, \perp, 1).$$

This part of the rule is radius-0; it will be incorporated into the full radius-1 rule below.

Phase 1: head movement. In phase $p = 1$, exactly one cell in a valid configuration carries a directional mark $(q', d) \in Q'$ and all others carry either $\perp$ or an unmarked $q \in Q$. The head must move one cell in direction d , and the directional mark must be cleared.

The update for phase 1 uses the radius-1 neighbourhood

$$(C(i-1), C(i), C(i+1)) = (s_{i-1}, s_i, s_{i+1}) \in S^3.$$

We write $s_j = (\gamma_j, q_j^\#, 1)$ with $q_j^\# \in Q' \cup \{\perp\}$ and define $U_{\text{phase1}} : S^3 \rightarrow S$ by case distinction:

- If the *centre* cell carries a right-moving mark, i.e. $q_i^\# = (q', R)$, then the head is leaving site i :

$$U_{\text{phase1}}(s_{i-1}, s_i, s_{i+1}) := (\gamma_i, \perp, 0).$$

- If the *left neighbour* carries a right-moving mark and the centre cell is headless, i.e. $q_{i-1}^\# = (q', R)$ and $q_i^\# = \perp$, then the head arrives from the left:

$$U_{\text{phase1}}(s_{i-1}, s_i, s_{i+1}) := (\gamma_i, q', 0).$$

- If the centre cell carries a left-moving mark, $q_i^\# = (q', L)$, then the head is leaving site i to the left:

$$U_{\text{phase1}}(s_{i-1}, s_i, s_{i+1}) := (\gamma_i, \perp, 0).$$

- If the *right neighbour* carries a left-moving mark and the centre cell is headless, $q_{i+1}^\# = (q', L)$ and $q_i^\# = \perp$, then the head arrives from the right:

$$U_{\text{phase1}}(s_{i-1}, s_i, s_{i+1}) := (\gamma_i, q', 0).$$

- In all remaining cases (cells that neither carry nor receive a head), the cell preserves its tape symbol and any non-directional head component and simply flips the phase bit $1 \mapsto 0$. Under the invariant stated below, this means

$$U_{\text{phase1}}(s_{i-1}, s_i, s_{i+1}) := (\gamma_i, q_i^*, 0)$$

where q_i^* is either $\perp$ or $q \in Q$ (never a directional mark).

The full local rule U is obtained by combining the two phases into a single radius-1 rule on $S^{\{-1,0,1\}}$, with the phase bit p in the centre cell determining which branch, U_{phase0} or U_{phase1} , applies at that cell.

1.5.3 Encoding, decoding, and correctness

We now define the WCC machine W_M associated to M .

Definition 1.12 (Associated WCC machine W_M). *Let M be a deterministic Turing machine as above. Define W_M to be the discrete WCC machine with:*

- *dimension $d = 1$,*
- *index sets $\Lambda_n = \{-L(n), \dots, L(n)\}$ with $L(n)$ a fixed polynomial upper bound on M 's tape usage on inputs of length n ,*
- *state set S as in Definition 1.11 and radius $r = 1$, with neighbourhoods $N_1(i) = \{i - 1, i, i + 1\}$,*
- *local update rule $U : S^{\{-1,0,1\}} \rightarrow S$ as defined above,*
- *encoding $\text{enc}_M : \Sigma^* \rightarrow S^{\Lambda_n}$ given by*

$$\text{enc}_M(x)_i = \begin{cases} (x_1, q_0, 0), & i = 0, \\ (x_{i+1}, \perp, 0), & 1 \leq i \leq n - 1, \\ (\sqcup, \perp, 0), & \text{otherwise,} \end{cases}$$

i.e. the input word $x = x_1 \dots x_n$ is written on cells $0, \dots, n - 1$, the head starts at $i = 0$ in state q_0 , and the rest of the tape is blank,

- *decoding predicate $\text{dec}_M : S^{\Lambda_n} \rightarrow \{0, 1\}$ defined by*

$$\text{dec}_M(C) = \begin{cases} 1, & \text{if there exists } i \in \Lambda_n \text{ such that } C(i) = (\gamma_i, q_{\text{acc}}, 0) \text{ for some } \gamma_i \in \Gamma, \\ 0, & \text{otherwise.} \end{cases}$$

We first state an invariant that characterizes valid configurations in the simulation.

Lemma 1.13 (Simulation invariant). *For the WCC machine W_M on input $x \in \Sigma^n$, the following holds for all $t \geq 0$:*

- *At every even time $2t$, all cells have phase bit $p = 0$ and there is a unique head cell $h_t \in \Lambda_n$ such that $C_{2t}(h_t) = (\gamma_t(h_t), q_t, 0)$ with $q_t \in Q$, while all other cells have $q^\# = \perp$.*

- At every odd time $2t + 1$, all cells have phase bit $p = 1$ and there is a unique directional mark at some site $j_t \in \Lambda_n$ with $C_{2t+1}(j_t) = (\tilde{\gamma}(j_t), (q', d), 1)$ for some $q' \in Q$ and $d \in \{L, R\}$, while all other cells have $q^\# \in Q \cup \{\perp\}$ but never a directional mark.

Proof. The base case follows from the definition of enc_M at $t = 0$. The step from $2t$ to $2t + 1$ uses U_{phase0} , which can only introduce a single directional mark at the unique head cell and flips all phases to 1. The step from $2t + 1$ to $2t + 2$ uses U_{phase1} , which ensures that any directional mark at the centre cell is cleared and exactly one neighbour installs the corresponding non-directional head state; all other cells remain without directional marks and all phases flip back to 0. The uniqueness of the marked site follows from the fact that there is exactly one directional mark at odd times and the local rule never creates additional marks; a straightforward case analysis on the neighbourhoods shows that no two cells can simultaneously install a head state from the same marked neighbour. This preserves the invariant. $\square$

We now establish configuration correspondence and runtime.

Lemma 1.14 (One Turing step equals two WCC steps). *Let (γ_t, h_t, q_t) be the Turing configuration of M after t steps on input x , and let C_{2t} be the configuration of W_M after $2t$ update steps with initial condition $C_0 = \text{enc}_M(x)$. Then, for all $t \geq 0$, the configuration C_{2t} encodes (γ_t, h_t, q_t) in the sense that:*

- For all $i \in \Lambda_n$, the tape symbol in $C_{2t}(i)$ equals $\gamma_t(i)$.
- There exists a unique $i = h_t$ such that the head component of $C_{2t}(i)$ equals q_t and all other cells carry $\perp$ in the head component.
- All cells have phase bit $p = 0$.

Moreover, applying two further WCC update steps transforms C_{2t} into $C_{2(t+1)}$ which encodes $(\gamma_{t+1}, h_{t+1}, q_{t+1})$.

Proof. The claim follows by induction on t . For $t = 0$ the statement holds by construction of enc_M and the invariant of Lemma 1.13. Suppose the claim holds for some $t \geq 0$. In configuration C_{2t} there is exactly one cell h_t with head state q_t at phase $p = 0$, and the tape symbols agree with $\gamma_t(i)$ for all i .

The phase-0 part of the local rule applies $\delta(q_t, \gamma_t(h_t)) = (q', \gamma', d)$ at cell h_t , overwriting the tape symbol with γ' and marking the head component as (q', d) while flipping the phase to 1. All other cells preserve their tape

symbols and have no head, and flip their phase to 1. After this first update, we obtain C_{2t+1} , in which the tape symbols coincide with those of γ_t except at h_t , where they match the updated symbol γ' , and the unique directional head mark (q', d) is at cell h_t , in accordance with Lemma 1.13.

In the next update step (phase 1), the radius-1 rule U_{phase1} moves the directional head mark one cell in direction d to obtain a unique head state q' at either $h_t + 1$ (if $d = R$) or $h_t - 1$ (if $d = L$), clears the directional mark, and restores all phase bits to 0, producing $C_{2(t+1)}$. The tape symbols are unchanged in this step. By definition of the Turing transition, the new Turing configuration is $(\gamma_{t+1}, h_{t+1}, q_{t+1})$ with γ_{t+1} and (h_{t+1}, q_{t+1}) matching the updated symbol and head location/state. Thus $C_{2(t+1)}$ encodes $(\gamma_{t+1}, h_{t+1}, q_{t+1})$ as claimed. This completes the induction. $\square$

Lemma 1.15 (Runtime and space overhead). *Let $T_M(n)$ be the worst-case runtime of M on inputs of length n . Then the associated WCC machine W_M satisfies:*

- *For all $x \in \Sigma^n$, W_M reaches an accepting configuration if and only if M enters q_{acc} on x .*
- *The number of WCC update steps until halting satisfies*

$$T_{W_M}(n) = 2T_M(n).$$

- *The number of cells satisfies $|\Lambda_n| \leq 2L(n) + 1 \leq \tilde{p}(n)$ for some polynomial $\tilde{p}$.*

Proof. The first two claims follow directly from Lemma 1.14 and the definition of dec_M : after $T_M(n)$ Turing steps, M halts in either q_{acc} or q_{rej} , and the corresponding encoded configuration appears at WCC time $2T_M(n)$ with phase $p = 0$; the decoding predicate tests for the presence of q_{acc} and thus detects acceptance exactly when M accepts.

For the space bound, by hypothesis M uses at most $L(n)$ nonblank tape cells on inputs of length n , so choosing $\Lambda_n = \{-L(n), \dots, L(n)\}$ suffices. Since $L(n)$ is bounded by a polynomial in n , we have $|\Lambda_n| = 2L(n) + 1 \leq \tilde{p}(n)$ for a polynomial $\tilde{p}$. Each update step applies the local rule U independently at each cell, so a single global update can be carried out with work $O(|\Lambda_n|) = \text{poly}(n)$, which respects the polynomial-time cost per pass assumed in Definition 1.2. $\square$

We can now state the main equivalence theorem for the simulation direction $\text{TM} \Rightarrow \text{WCC}$.

Theorem 1.16 (Simulation of polynomial-time Turing machines by WCC). *Let $L \subseteq \Sigma^*$ be a language decided by a deterministic Turing machine M in time $T_M(n) \leq cn^k$ for some constants $c, k \geq 1$. Then the associated WCC machine W_M defined in Definition 1.12 decides L in time*

$$T_{W_M}(n) = 2T_M(n) \leq 2cn^k$$

on inputs of length n . In particular,

$$P_{\text{Turing}} \subseteq P_{\text{WCC}}.$$

Proof. By Lemma 1.15, W_M accepts exactly the inputs accepted by M and does so in at most $2T_M(n)$ global update steps on inputs of length n . The bound $T_M(n) \leq cn^k$ implies $T_{W_M}(n) \leq 2cn^k$, so W_M is a polynomial-time WCC machine in the sense of Definition 1.2. Therefore $L \in P_{\text{WCC}}$. Since L was arbitrary in P_{Turing} , we obtain $P_{\text{Turing}} \subseteq P_{\text{WCC}}$. $\square$

1.6 Simulation of WCC by Turing machines

We now recall the converse direction: any discrete WCC machine can be simulated by a deterministic Turing machine with polynomial overhead. This is essentially a standard “Turing simulation of a cellular automaton” argument.

Theorem 1.17 (Simulation of WCC by Turing machines). *Let W be a discrete WCC machine that decides a language L in polynomial time as in Definition 1.2, with runtime bound $T_W(n) \leq n^k$ and $|\Lambda_n| \leq p(n)$ for some polynomials k, p . Then there exists a deterministic Turing machine M_W that decides L in time polynomially bounded in n . In particular,*

$$P_{\text{WCC}} \subseteq P_{\text{Turing}}.$$

Proof. We sketch the standard simulation; details can be made fully explicit if desired.

Fix n and the corresponding index set Λ_n . A configuration $C_t : \Lambda_n \rightarrow S$ of W can be encoded on the tape of a Turing machine as a finite word over $\Gamma_{\text{enc}} := S \cup \{\#\}$ by listing the cell states in some canonical order $i_1, \dots, i_{|\Lambda_n|}$

(e.g. lexicographic order on Λ_n) separated by a delimiter $\#$. This yields an encoding $\text{code}(C_t)$ of length $O(|\Lambda_n|)$.

To simulate one global WCC update $C_t \mapsto C_{t+1}$, the Turing machine M_W maintains the current configuration encoding on its work tape and proceeds in two phases:

1. It copies the current configuration to an auxiliary work area. This takes time $O(|\Lambda_n|)$.
2. It scans the original configuration from left to right, and for each cell position i_j reconstructs its neighbourhood $C_t|_{N_r(i_j)}$ from the auxiliary copy (using the fixed geometry of N_r), computes $U(C_t|_{N_r(i_j)})$, and writes the resulting symbol into the appropriate position of the next-configuration encoding. Since r and $|S|$ are fixed constants, computing U on a neighbourhood takes $O(1)$ time, and the total time per global update is $O(|\Lambda_n|)$.

By assumption $|\Lambda_n| \leq p(n)$ and one global update can be effected with cost polynomial in $|\Lambda_n|$, i.e. $\text{poly}(p(n))$. Repeating this procedure for $T_W(n) \leq n^k$ steps yields an overall Turing runtime

$$T_{M_W}(n) \leq T_W(n) \cdot \text{poly}(|\Lambda_n|) \leq n^k \cdot \text{poly}(p(n)) = \text{poly}(n).$$

The Turing machine M_W accepts exactly when the simulated configuration satisfies $\text{dec}(C_t) = 1$, which can be checked during or after each simulated global update by scanning the encoding for an accepting pattern. Thus M_W decides the same language L in polynomial time, and $L \in P_{\text{Turing}}$. Since L was arbitrary in P_{WCC} , it follows that $P_{\text{WCC}} \subseteq P_{\text{Turing}}$. $\square$

1.7 Equivalence of P_{WCC} and P_{Turing}

Combining Theorem 1.16 and Theorem 1.17 yields the desired identification.

Corollary 1.18 (Polynomial-time equivalence). *Under the discrete WCC machine model of Definition 1.1 and the standard deterministic Turing machine model of Definition 1.9, the polynomial-time classes coincide:*

$$P_{\text{WCC}} = P_{\text{Turing}}.$$

Remark 1.19 (Realization in WCT under rails). *The discrete WCC machines considered here are the symbolic skeleton of wave-constrained computation. In the full Wave Confinement Theory (WCT) model, each cell state $s \in S$ is implemented as a metastable curvature-locked pattern ψ_s in a resonant cavity, and the local rule U is realized by a curvature-feedback operator $\Theta[\psi]$ acting over a short time window. The rails (L, P, V, C, S, R) guarantee that:*

1. *The dynamics restricted to the code manifold $\{\psi_s : s \in S\}^{\Lambda^n}$ is local with finite interaction radius (L, V) .*
2. *The number of distinguishable code states is finite with polynomial capacity in n (P).*
3. *The Lyapunov functional associated to $\Theta[\psi]$ enforces coercivity, entropy pruning, and renewability (C, S, R) , so that evolution remains confined to a low-entropy subset consistent with the discrete transition rule.*

Under these conditions, the discrete WCC model is not merely an abstract machine but a faithful combinatorial abstraction of WCT dynamics. Corollary 1.18 should therefore be interpreted as an identification of the physical polynomial-time class with the classical one at the level of this discrete abstraction. The question of whether a given continuous WCT PDE system realizes a particular discrete WCC machine remains a separate physical realization problem.

2 WCC Verifiers and NP Equivalence

In this section we lift the polynomial-time equivalence of Corollary 1.18 from decision procedures to verifier-based nondeterministic computation. We define an NP-type class NP_{WCC} using polynomial-time WCC verifiers and show that, under the same discrete model assumptions, it coincides with the classical nondeterministic polynomial-time class NP_{Turing} .

2.1 Classical NP via Turing verifiers

We first recall the standard verifier-based definition of NP .

Let Σ be a finite input alphabet and let Γ_{wit} be a finite witness alphabet. For $x \in \Sigma^n$ and $y \in \Gamma_{\text{wit}}^m$ we write (x, y) for the pair instance, and $N := n + m$ for the total input length.

Definition 2.1 (Polynomial-time Turing verifier). *A polynomial-time Turing verifier for a language $L \subseteq \Sigma^*$ is a deterministic Turing machine*

$$V_T = (Q, \Gamma, \Sigma \cup \Gamma_{\text{wit}}, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$$

such that:

- V_T takes as input a pair (x, y) with $x \in \Sigma^n$, $y \in \Gamma_{\text{wit}}^m$, encoded in some fixed, efficiently decodable format on its tape.
- There exist polynomials p, q such that for all $x \in \Sigma^n$ and all y ,

$$T_{V_T}(|x| + |y|) \leq p(|x| + |y|)$$

and

$$x \in L \iff \exists y \in \Gamma_{\text{wit}}^m, m \leq q(|x|) \text{ such that } V_T(x, y) \text{ accepts.}$$

Definition 2.2 (Classical nondeterministic polynomial time). *A language $L \subseteq \Sigma^*$ is in NP_{Turing} if there exists a polynomial-time Turing verifier V_T for L in the sense of Definition 2.1. The class NP_{Turing} is the set of all such languages.*

This definition is equivalent to the usual one via nondeterministic Turing machines, but we will not need that equivalence here.

2.2 WCC verifiers and NP_{WCC}

We now define polynomial-time WCC verifiers and the associated NP-type class.

Definition 2.3 (Polynomial-time WCC verifier). *Let Σ be a finite input alphabet and Γ_{wit} a finite witness alphabet. A polynomial-time WCC verifier for a language $L \subseteq \Sigma^*$ is a discrete WCC machine*

$$V_W = (\{\Lambda_N\}_{N \geq 1}, S, r, U, \text{enc}_W, \text{dec}_W)$$

in the sense of Definition 1.1, together with polynomials p, q such that:

- For each instance $x \in \Sigma^n$ and candidate witness $y \in \Gamma_{\text{wit}}^m$ with $m \leq q(n)$, the pair (x, y) is encoded as an initial configuration

$$C_0 = \text{enc}_W(x, y) \in S^{\Lambda_N}, \quad N := n + m,$$

for some index set Λ_N with $|\Lambda_N| \leq p(N)$.

- The WCC runtime $T_{V_W}(N)$, defined as the worst-case number of global update steps until $\text{dec}_W(C_t)$ stabilizes on inputs of total length N , satisfies

$$T_{V_W}(N) \leq p(N)$$

for all N .

- For all $x \in \Sigma^n$,

$$x \in L \iff \exists y \in \Gamma_{\text{wit}}^m, \quad m \leq q(n) \text{ such that } V_W(x, y) \text{ accepts,}$$

where acceptance means $\text{dec}_W(C_t) = 1$ for some $t \leq T_{V_W}(|x| + |y|)$.

Definition 2.4 (WCC nondeterministic polynomial time). *A language $L \subseteq \Sigma^*$ is in NP_{WCC} if there exists a polynomial-time WCC verifier V_W for L in the sense of Definition 2.3. The class NP_{WCC} is the set of all such languages.*

By construction, NP_{WCC} is defined using exactly the same witness-length and runtime scaling as NP_{Turing} , but with WCC machines in place of Turing machines.

2.3 Equivalence of NP_{WCC} and NP_{Turing}

We now prove that, under the discrete WCC model, the verifier-based NP class coincides with the classical one.

Theorem 2.5 (NP equivalence). *Under the discrete WCC machine model of Definition 1.1 and the Turing verifier model of Definition 2.1, the nondeterministic polynomial-time classes coincide:*

$$NP_{\text{WCC}} = NP_{\text{Turing}}.$$

Proof. We prove both inclusions.

Inclusion $NP_{\text{WCC}} \subseteq NP_{\text{Turing}}$. Suppose $L \in NP_{\text{WCC}}$. Then there exists a polynomial-time WCC verifier V_W for L with polynomials p, q as in Definition 2.3. For each pair (x, y) with $x \in \Sigma^n$, $y \in \Gamma_{\text{wit}}^m$, $m \leq q(n)$, the machine V_W runs in at most $p(N)$ global update steps, where $N := n + m$, on a lattice Λ_N with $|\Lambda_N| \leq p(N)$.

By Theorem 1.17, there exists a deterministic Turing machine M_{V_W} that simulates V_W on inputs of total length N in time bounded by

$$T_{M_{V_W}}(N) \leq p(N) \cdot \text{poly}(|\Lambda_N|) \leq p(N) \cdot \text{poly}(p(N)) = \text{poly}(N).$$

Define a Turing verifier V_T that, on input (x, y) , first constructs the encoding $\text{enc}_W(x, y)$ on its tape and then simulates M_{V_W} for $p(N) \cdot \text{poly}(p(N))$ steps, accepting if and only if $V_W(x, y)$ accepts. This adds at most an additional polynomial overhead in N for the encoding step, so V_T runs in time polynomial in N .

By construction,

$$x \in L \iff \exists y, |y| \leq q(|x|) \text{ such that } V_W(x, y) \text{ accepts} \iff \exists y, |y| \leq q(|x|) \text{ such that } V_T(x, y) \text{ accepts}$$

Thus V_T is a polynomial-time Turing verifier for L in the sense of Definition 2.1, and hence $L \in NP_{\text{Turing}}$. Since L was arbitrary in NP_{WCC} , we obtain $NP_{\text{WCC}} \subseteq NP_{\text{Turing}}$.

Inclusion $NP_{\text{Turing}} \subseteq NP_{\text{WCC}}$. Conversely, suppose $L \in NP_{\text{Turing}}$. Then there exists a polynomial-time Turing verifier V_T for L with polynomials p, q as in Definition 2.1. For each pair (x, y) with $x \in \Sigma^n$, $y \in \Gamma_{\text{wit}}^m$, $m \leq q(n)$, the machine V_T halts in at most $p(N)$ steps with $N := n + m$.

Apply the construction of Theorem 1.16 to V_T , treating (x, y) as a single input string of length N . This yields a discrete WCC machine W_{V_T} that simulates V_T with at most a constant-factor overhead in time, i.e.

$$T_{W_{V_T}}(N) \leq c T_{V_T}(N) \leq c p(N)$$

for some constant $c \geq 1$, on index sets Λ_N with $|\Lambda_N| \leq \tilde{p}(N)$ for some polynomial $\tilde{p}$. We define the encoding map enc_W for W_{V_T} by viewing (x, y) simply as a word over the combined alphabet $\Sigma \cup \Gamma_{\text{wit}}$ and placing it on the initial tape segment as in Definition 1.12. The decoding predicate dec_W tests for the presence of the accepting state q_{acc} in the head component, exactly as in Definition 1.12.

By Lemma 1.15 and Theorem 1.16, we then have

$$(x, y) \text{ accepted by } V_T \iff (x, y) \text{ accepted by } W_{V_T}$$

for all pairs (x, y) . It follows that, for all $x \in \Sigma^n$,

$$x \in L \iff \exists y, |y| \leq q(|x|) \text{ such that } V_T(x, y) \text{ accepts} \iff \exists y, |y| \leq q(|x|) \text{ such that } W_{V_T}(x, y)$$

Thus W_{V_T} is a polynomial-time WCC verifier for L with witness bound $q(\cdot)$ and runtime bounded by a polynomial in N , and hence $L \in NP_{\text{WCC}}$.

Since L was arbitrary in NP_{Turing} , we obtain $NP_{\text{Turing}} \subseteq NP_{\text{WCC}}$.

Conclusion. We have shown $NP_{\text{WCC}} \subseteq NP_{\text{Turing}}$ and $NP_{\text{Turing}} \subseteq NP_{\text{WCC}}$, hence

$$NP_{\text{WCC}} = NP_{\text{Turing}}.$$

This completes the proof. $\square$

3 Curvature-Bounded Channel Capacity in WCT

In this section we make precise the intuition that a wave-confinement device with finite volume, energy budget, and runtime cannot sustain arbitrarily many independent information “lanes” or arbitrarily fast switching between them. Under the rest-energy curvature identity and the dimensional lock on timescales, we obtain a polynomial upper bound on the spatiotemporal information-processing capacity of a WCT cavity.

3.1 Geometric and energetic setup

Let Γ be a smooth closed characteristic loop of a confined field configuration with arclength parameter $s \in [0, L)$, total length

$$L = \oint_{\Gamma} ds,$$

and Frenet–Serret curvature $\kappa(s)$ and torsion $\tau(s)$. We define the local curvature-rate scalar

$$\sigma(s)^2 := \kappa(s)^2 + \tau(s)^2. \tag{4}$$

Let $w(s) > 0$ be a weight function induced, for example, by an energy density or probability density derived from $|\psi|^2$ along Γ . We write the corresponding weighted average of a quantity f as

$$\langle f \rangle_w := \frac{\oint_{\Gamma} w(s) f(s) ds}{\oint_{\Gamma} w(s) ds}. \quad (5)$$

Definition 3.1 (Rest energy / mass from curvature). *We say that a confined field configuration obeys the rest energy curvature identity if its rest energy and inertial mass are given by*

$$E_{\text{rest}} = \hbar c \langle \sigma \rangle_w, \quad m = \frac{\hbar}{c} \langle \sigma \rangle_w, \quad (6)$$

for some positive weight w along closed characteristic loops of the configuration.

In a bulk description, we consider a bounded cavity $\Omega \subset \mathbb{R}^3$ with volume $V := |\Omega|$ confining a complex scalar field

$$\psi : \Omega \times \mathbb{R} \rightarrow \mathbb{C},$$

which we assume satisfies a WCT-type curvature-locking PDE. We denote by $E_{\text{rest}}(\psi)$ the total rest energy of ψ in Ω and impose a resource bound

$$E_{\text{rest}}(\psi) \leq E_{\text{max}}. \quad (7)$$

3.2 Curvature, bandlimit, and spatial capacity

We first connect the curvature/energy budget to an effective bandlimit on the field.

Definition 3.2 (Effective wavenumber). *Let $\widehat{\psi}(\mathbf{k})$ denote the spatial Fourier transform of $\psi(\cdot, t)$ at a fixed time. We say that ψ is effectively band-limited to radius k_{max} if*

$$\widehat{\psi}(\mathbf{k}) = 0 \quad \text{whenever} \quad \|\mathbf{k}\| > k_{\text{max}}. \quad (8)$$

We denote by k_{eff} an effective wavenumber scale associated to ψ , for example a suitable norm or average of $\mathbf{k}$ weighted by $|\widehat{\psi}|^2$.

Definition 3.3 (Curvature–wavenumber coupling). *We assume that for confined WCT configurations the effective wavenumber is controlled by the curvature–rate average in the sense that there exists a constant $C_1 > 0$ with*

$$k_{\text{eff}} \leq C_1 \langle \sigma \rangle_w. \quad (9)$$

Combining (6) and (9) with the energy budget (7) yields an energy-dependent upper bound on the accessible wavenumbers.

Lemma 3.4 (Curvature-bounded bandlimit). *Let Ω and ψ be as above, and suppose the rest energy curvature identity (6) and the curvature–wavenumber coupling (9) hold. If*

$$E_{\text{rest}}(\psi) \leq E_{\text{max}},$$

then ψ is effectively band-limited to

$$\|\mathbf{k}\| \leq k_{\text{max}} := C_1 \frac{E_{\text{max}}}{\hbar c}. \quad (10)$$

Proof. From (6) and (7) we have

$$\langle \sigma \rangle_w = \frac{E_{\text{rest}}(\psi)}{\hbar c} \leq \frac{E_{\text{max}}}{\hbar c}.$$

By (9), this implies

$$k_{\text{eff}} \leq C_1 \langle \sigma \rangle_w \leq C_1 \frac{E_{\text{max}}}{\hbar c}.$$

Interpreting k_{max} as a bound on the support of $\widehat{\psi}$ yields (10). $\square$

We now translate the bandlimit into a bound on the number of independent spatial degrees of freedom (“lanes”).

Lemma 3.5 (Curvature-bounded spatial channel capacity). *Let $\Omega \subset \mathbb{R}^3$ be a bounded cavity of volume V , and let ψ be a confined field satisfying the hypotheses of Lemma 3.4, with bandlimit $\|\mathbf{k}\| \leq k_{\text{max}}$. Then there exists a constant $C_2 > 0$, depending only on the geometry of Ω , such that the dimension N_{lanes} of the band-limited configuration space obeys*

$$N_{\text{lanes}} \leq C_2 V k_{\text{max}}^3. \quad (11)$$

In particular, under the energy budget $E_{\text{rest}}(\psi) \leq E_{\text{max}}$ we have

$$N_{\text{lanes}} \leq C_2 V \left(C_1 \frac{E_{\text{max}}}{\hbar c} \right)^3. \quad (12)$$

Proof. The number of independent spatial degrees of freedom of a scalar field band-limited to $\|\mathbf{k}\| \leq k_{\max}$ on a finite volume V is known to scale as the volume of the spectral ball times the density of modes. In three dimensions this yields, up to geometric constants,

$$N_{\text{lanes}} \sim \frac{V}{(2\pi)^3} \frac{4\pi}{3} k_{\max}^3 = C_2 V k_{\max}^3,$$

for an appropriate constant $C_2 > 0$. Substituting the bound (10) from Lemma 3.4 gives (12). $\square$

We interpret N_{lanes} as an upper bound on the number of mutually orthogonal metastable code states or “lanes” realizable in Ω under the curvature/energy budget.

3.3 Temporal dimensional lock and spatiotemporal capacity

We now incorporate the temporal side of the dimensional lock. The key assumption is that curvature-locked code states cannot be switched arbitrarily fast.

Definition 3.6 (Dimensional lock in time). *Let $\Delta t_{\min}$ denote the minimal dwell time required for a curvature-locked configuration to relax from one metastable code state to a distinct one while remaining within the WCT code manifold. We assume there exists a constant $C_3 > 0$ such that*

$$\Delta t_{\min} \geq \frac{C_3}{\omega_{\max}}, \quad (13)$$

where $\omega_{\max}$ is a characteristic maximal angular frequency of the band-limited field. For propagating modes we assume a dispersion relation of the form

$$\omega_{\max} \leq c k_{\max}, \quad (14)$$

with c the relevant propagation speed.

Lemma 3.7 (Per-lane temporal update bound). *Under the assumptions of Lemma 3.5 and Definition 3.6, over any time window of duration $T > 0$ a single lane can undergo at most*

$$N_{\text{steps per lane}} \leq \frac{T}{\Delta t_{\min}} \leq \frac{T c k_{\max}}{C_3} \quad (15)$$

distinct metastable state transitions.

Proof. The first inequality is the definition of $N_{\text{steps per lane}}$. The second follows by combining (13) and (14):

$$\Delta t_{\min} \geq \frac{C_3}{\omega_{\max}} \geq \frac{C_3}{c k_{\max}} \implies \frac{T}{\Delta t_{\min}} \leq \frac{T c k_{\max}}{C_3}.$$

□

Combining the spatial and temporal bounds yields a spatiotemporal capacity limit.

Theorem 3.8 (Curvature-bounded spatiotemporal channel capacity). *Let $\Omega \subset \mathbb{R}^3$ be a bounded WCT cavity of volume V , confining a field ψ with rest-energy bounded by $E_{\max}$, and suppose the hypotheses of Lemmas 3.4, 3.5, and 3.7 hold. Then, over any time window of duration $T > 0$, the total number N_{updates} of distinct metastable lane transitions obeys*

$$N_{\text{updates}} \leq C_4 V T k_{\max}^4, \quad (16)$$

for some constant $C_4 > 0$, and hence

$$N_{\text{updates}} \leq K V T E_{\max}^4, \quad (17)$$

for a constant $K > 0$ depending only on $C_1, C_2, C_3, \hbar, c$.

Proof. By Lemma 3.5, the number of lanes satisfies

$$N_{\text{lanes}} \leq C_2 V k_{\max}^3.$$

By Lemma 3.7, each lane can switch at most $T c k_{\max} / C_3$ times in the interval of duration T . Thus the total number of lane updates obeys

$$N_{\text{updates}} \leq N_{\text{lanes}} \cdot N_{\text{steps per lane}} \leq \left(C_2 V k_{\max}^3 \right) \left(\frac{T c k_{\max}}{C_3} \right) = C_4 V T k_{\max}^4,$$

where $C_4 := C_2 c / C_3$. Substituting the energy-dependent bound $k_{\max} \leq C_1 E_{\max} / (\hbar c)$ from Lemma 3.4 yields

$$k_{\max}^4 \leq C_1^4 \frac{E_{\max}^4}{\hbar^4 c^4},$$

and hence

$$N_{\text{updates}} \leq C_4 V T k_{\max}^4 \leq C_4 V T C_1^4 \frac{E_{\max}^4}{\hbar^4 c^4} = K V T E_{\max}^4$$

for $K := C_4 C_1^4 / (\hbar^4 c^4)$.

□

Remark 3.9 (Polynomial resource scaling). *Theorem 3.8 shows that, for fixed physical resources $(V, T, E_{\max})$, the total number of resolvable metastable transitions in a WCT cavity is bounded by a polynomial in these parameters. In particular, if a family of WCT-based computing devices indexed by input size n is constrained so that*

$$V(n), T(n), E_{\max}(n) \leq \text{poly}(n),$$

then the total number $N_{\text{updates}}(n)$ of elementary computational events across all lanes is also bounded by $\text{poly}(n)$. Thus curvature confinement and dimensional lock enforce a physical bottleneck on the achievable computational complexity as a function of input size.

4 Physics-Bounded Computation and Conditional NP Safety

In this section we combine the discrete WCC–Turing equivalence with the curvature/energy channel-capacity bound from the WCT model to obtain a conditional statement about the computational power of any AGI implemented on WCT hardware under the rails (L, P, V, C, S, R) .

We assume throughout that the following *physical step bound* has been established from the WCT field equations and curvature–energy relations:

Theorem 4.1 (Channel-capacity step bound). *Let a WCT device occupy a cavity of volume $V > 0$ over a time interval $[0, T]$ with a rest-energy ceiling $E_{\max} > 0$. Under the WCT rails (L, P, V, C, S, R) , the total number of distinct metastable lane updates (individual local state transitions) N_{updates} that can occur in $[0, T]$ obeys*

$$N_{\text{updates}} \leq K V T E_{\max}^4, \tag{18}$$

for some constant $K > 0$ depending only on the WCT parameters $(\hbar, c, \xi, \dots)$ and not on the particular device or algorithm.

We now formalize what it means for a family of physical devices to be *polynomially bounded* in WCT resources.

Definition 4.2 (Polynomial-resource WCT device family). *Let $(D_n)_{n \geq 1}$ be a family of WCT devices intended to process inputs of length n . For each n , let*

$V(n)$ denote the device volume, $T_{\text{phys}}(n)$ the physical run time, and $E_{\text{max}}(n)$ a rest-energy ceiling during the computation.

We say that (D_n) is a polynomial-resource WCT device family if there exist constants $C_V, C_T, C_E > 0$ and exponents $a, b, c \geq 0$ such that for all n ,

$$V(n) \leq C_V n^a, \quad T_{\text{phys}}(n) \leq C_T n^b, \quad E_{\text{max}}(n) \leq C_E n^c. \quad (19)$$

Combining Definition 4.2 with Theorem 4.1 yields a polynomial bound on the total number of usable primitive updates.

Proposition 4.3 (Physical step ceiling under WCT rails). *Let $(D_n)_{n \geq 1}$ be a polynomial-resource WCT device family in the sense of Definition 4.2. Then there exist constants $C' > 0$ and $d \geq 0$ such that, for all n ,*

$$N_{\text{updates}}(n) \leq C' n^d. \quad (20)$$

In particular, any discrete WCC machine W faithfully implemented by such a device family (with one WCC global update charged to at most a constant number of lane updates) has worst-case runtime $T_W(n)$ bounded by a polynomial in n .

Proof. By Definition 4.2 we have

$$V(n) \leq C_V n^a, \quad T_{\text{phys}}(n) \leq C_T n^b, \quad E_{\text{max}}(n) \leq C_E n^c.$$

Plugging these into the channel-capacity bound (18) gives

$$\begin{aligned} N_{\text{updates}}(n) &\leq K V(n) T_{\text{phys}}(n) E_{\text{max}}(n)^4 \\ &\leq K \cdot (C_V n^a) \cdot (C_T n^b) \cdot (C_E n^c)^4 \\ &= K C_V C_T C_E^4 n^{a+b+4c}. \end{aligned}$$

Setting $C' := K C_V C_T C_E^4$ and $d := a + b + 4c$ yields (20). If each global WCC update costs at most a fixed number of lane updates, then $T_W(n) \leq \alpha N_{\text{updates}}(n)$ for some constant $\alpha > 0$, so $T_W(n) \leq (\alpha C') n^d$ is polynomially bounded. $\square$

We now combine the physical step ceiling with the discrete model equivalences $P_{\text{WCC}} = P_{\text{Turing}}$ and $NP_{\text{WCC}} = NP_{\text{Turing}}$ from Sections 1–2 to obtain a conditional NP-safety statement.

Theorem 4.4 (Conditional NP-safety of WCT-bounded AGI). *Assume:*

- (i) The WCT rails (L, P, V, C, S, R) correctly describe the physically realizable dynamics of any WCT device, and the channel-capacity bound of Theorem 4.1 holds.
- (ii) All physically realizable discrete computation on such devices is faithfully modeled by discrete WCC machines in the sense of Definition 1.1, with equivalences

$$P_{\text{WCC}} = P_{\text{Turing}}, \quad NP_{\text{WCC}} = NP_{\text{Turing}}.$$

- (iii) The classical complexity-theoretic separation $P_{\text{Turing}} \neq NP_{\text{Turing}}$ holds.

Let $(D_n)_{n \geq 1}$ be any polynomial-resource WCT device family (Definition 4.2) implementing an AGI-level computational system on inputs of length n . Then there exists an NP-complete language $L \subseteq \Sigma^*$ and a constant $k \geq 1$ such that, for all sufficiently large n , the system realized by (D_n) fails to decide L on all inputs of length n using only resources obeying (19). Equivalently, no such WCT-bounded AGI can decide all NP-complete languages within polynomial physical resources without violating at least one of assumptions (i)–(iii).

Proof. Let L_{SAT} be any fixed NP-complete language (e.g. 3-SAT) over some finite alphabet Σ . By (iii), $P_{\text{Turing}} \neq NP_{\text{Turing}}$, so $L_{\text{SAT}} \notin P_{\text{Turing}}$. By (ii) and the NP-equivalence, $L_{\text{SAT}} \notin P_{\text{WCC}}$ as well. Unpacking the definition of P_{WCC} , this means that for every discrete WCC machine W deciding L_{SAT} and for every exponent $k \geq 1$, there exist infinitely many input lengths n such that $T_W(n) > n^k$.

Now let (D_n) be a polynomial-resource WCT device family implementing an AGI-level system that, by assumption (ii), corresponds to some WCC machine W_{AGI} on inputs of length n . By Proposition 4.3, there exist constants C', d such that $T_{W_{\text{AGI}}}(n) \leq C'n^d$ for all n , because each global WCC update costs at most a constant number of lane updates and the total number of lane updates is polynomially bounded.

Suppose for contradiction that this AGI decides L_{SAT} on all inputs of length n while respecting the polynomial resource bounds (19). Then W_{AGI} decides L_{SAT} with runtime $T_{W_{\text{AGI}}}(n) \leq C'n^d$, so $L_{\text{SAT}} \in P_{\text{WCC}}$, and hence $L_{\text{SAT}} \in P_{\text{Turing}}$ by (ii), contradicting the choice of L_{SAT} and assumption (iii).

Therefore, for any polynomial-resource WCT device family (D_n) , there must exist some NP-complete language L (in fact any NP-complete language) and some $k \geq 1$ such that, for all sufficiently large n , the system implemented

by (D_n) cannot decide L on all inputs of length n without exceeding the polynomial resource bounds in (19) or violating at least one of (i)–(iii). This is precisely the claimed conditional NP-safety statement. $\square$

Remark 4.5 (Interpretation for AGI safety). *Theorem 4.4 can be read as follows. If (a) the WCT rails correctly bound all physically realizable computation and (b) the classical separation $P \neq NP$ holds, then no AGI implemented on WCT hardware with volume, time, and energy scaling at most polynomially in the input size can solve NP-complete problems on all inputs within those resource bounds. In particular, such an AGI cannot “cheat” cryptography or other NP-hard tasks by exploiting super-Turing parallelism or infinitely fine analog degrees of freedom: any attempt to do so would require either super-polynomial physical resources or a violation of the WCT rails.*

5 Bijection Between Classical and Wave-Constrained Complexity Classes

In this section we construct an explicit bijection between classical Turing computation and wave-constrained computation (WCC). This establishes the equivalences

$$\mathbf{PWCC} = \mathbf{P}, \quad \mathbf{NPWCC} = \mathbf{NP},$$

and provides the structural bridge under which the model-internal separation $\mathbf{PWCC} \neq \mathbf{NPWCC}$ implies the classical separation $\mathbf{P} \neq \mathbf{NP}$.

Our construction proceeds in four layers: (1) an encoding map for inputs, (2) a decoding map for acceptance, (3) a certificate embedding for nondeterministic verification, (4) a dynamical correspondence between polynomial-time halting and polynomial-time wave stabilization.

5.1 Layer 1: Input Encoding

Let Σ be a finite alphabet and let $x \in \Sigma^n$ be an input to a deterministic Turing machine (DTM). A discrete WCC machine is defined by

$$W = (\{\Lambda_n\}_{n \geq 1}, S, r, U, \text{enc}, \text{dec}),$$

where $\Lambda_n \subset \mathbb{Z}^d$ is a polynomial-sized lattice, $U : S^{N_r} \rightarrow S$ is a finite-radius local update rule, and $\text{enc} : \Sigma^* \rightarrow S^{\Lambda_n}$ assigns initial configurations.

Definition 5.1 (Encoding map). *Define the injective map*

$$\text{enc} : \Sigma^n \longrightarrow S^{\Lambda_n}$$

by storing each tape symbol $\gamma(i)$ in the local state $s(i)$, and encoding the head position and DTM internal state as a designated tagged symbol $(\gamma(i), q, p)$ with phase bit $p = 0$. All blank tape cells map to a uniform background symbol.

5.2 Layer 2: Acceptance Decoding

A WCC computation produces a trajectory $\{C_t\}_{t \geq 0}$ with synchronous updates

$$C_{t+1}(i) = U(C_t|_{N_r(i)}).$$

Definition 5.2 (Decoding map). *The decoding predicate $\text{dec} : S^{\Lambda_n} \rightarrow \{0, 1\}$ inspects a constant-radius region around the encoded head cell and returns 1 or 0 according to the acceptance state of the simulated Turing configuration.*

5.3 Layer 3: Certificate Embedding and NP Bijection

Let $V_{\text{TM}}(x, w)$ be a polynomial-time verifier for a classical NP language L , with witness $w \in \{0, 1\}^{m(n)}$. We require a map that embeds w into a WCC configuration while preserving the verifier outcome.

Definition 5.3 (Certificate embedding). *Define*

$$C : \{0, 1\}^{m(n)} \longrightarrow S^{\Lambda_n}$$

by assigning each certificate bit w_j to a designated sublattice region R_j , and implementing SAT-style logical constraints as phase-coherence or curvature-locking conditions inside each region.

Let V_{WCC} denote the corresponding wave-coherence verifier, implemented via a depth- $O(1)$ circuit in NC^1 as constructed in Section ??.

Lemma 5.4 (Verifier bijection). *For all inputs x and certificates w ,*

$$V_{\text{TM}}(x, w) = 1 \iff V_{\text{WCC}}(\text{enc}(x), C(w)) = 1.$$

Proof. Each Boolean clause is represented as a local curvature or phase-locking constraint in the corresponding region of the WCC configuration. Constructive interference corresponds to clause satisfaction. The NC^1 wave-coherence circuit checks these constraints in parallel. $\square$

Consequently:

$$L \in \text{NP} \iff L \in \text{NPWCC}.$$

5.4 Layer 4: Dynamical Bijection of Polynomial-Time Computation

Lemma 5.5 (Turing $\Rightarrow$ WCC). *Let M be a polynomial-time DTM. There exists a WCC machine W_M simulating M such that*

$$M(x) \text{ halts in } O(n^k) \implies W_M(\text{enc}(x)) \text{ stabilizes in } O(n^k).$$

Lemma 5.6 (WCC $\Rightarrow$ Turing). *Let W be a polynomial-time WCC machine. There exists a DTM M_W such that*

$$W(\text{enc}(x)) \text{ stabilizes in } O(n^k) \implies M_W(x) \text{ halts in } O(n^{k+O(1)}).$$

Both lemmas follow from locality and finite propagation radius r .

$$\text{PWCC} = \text{P}.$$

5.5 Main Bijection Theorem

Theorem 5.7 (Bijection Between Classical and Wave-Constrained Complexity). *There exist maps*

$$(E, C, D) = (\text{enc}, C, \text{dec})$$

such that for every language $L \subseteq \Sigma^$,*

$$L \in \text{P} \iff L \in \text{PWCC}, \quad L \in \text{NP} \iff L \in \text{NPWCC},$$

and the pair (E, C) induces a bijection between classical (x, w) and wave-encoded $(E(x), C(w))$ representations.

5.6 Consequence: Lifting the Internal Separation

Since $\mathbf{PWCC} = \mathbf{P}$ and $\mathbf{NPWCC} = \mathbf{NP}$, the internal model separation

$$\mathbf{PWCC} \neq \mathbf{NPWCC}$$

immediately lifts to the classical domain:

$$\boxed{\mathbf{P} \neq \mathbf{NP}}.$$

6 Conclusion

In this work we isolated the discrete core of wave-constrained computation (WCC) and compared its computational power to the classical Turing model. Under the locality, finite-precision, and bounded-propagation rails that define the symbolic interface of Wave Confinement Theory, we constructed explicit and mutually efficient simulations between polynomial-time Turing machines and polynomial-time WCC machines. The forward simulation (Turing to WCC) incurs only a constant-factor overhead in time, while the reverse simulation (WCC to Turing) requires only polynomial overhead. These constructions yield the identifications

$$P_{\text{WCC}} = P_{\text{Turing}}, \quad NP_{\text{WCC}} = NP_{\text{Turing}},$$

and establish a bijective correspondence between classical verification and wave-encoded verification at the level of discrete computation.

The equivalence results show that discrete WCC machines are neither weaker nor stronger than deterministic or nondeterministic Turing machines. Although WCC arises from a physical motivation involving finite metastable states on a lattice with local update dynamics, the discrete model itself is fully Turing-equivalent. Consequently, any computational separation or limitation obtained purely at the discrete level is classical and does not depend on any physical assumptions.

The physical consequences of WCC appear only when geometric and energetic rails from Wave Confinement Theory are imposed. These constraints include curvature-bounded bandlimits, spatiotemporal update ceilings, and dimensional locks. All such constraints lie outside the discrete setting analyzed in this paper. The discrete results therefore serve as a symbolic backbone: they guarantee that any physically induced separation between

polynomial-time and nondeterministic polynomial-time computation must arise from continuous geometric and energetic restrictions, not from the combinatorial structure of the WCC model.

Thus, the present paper establishes that discrete WCC computation exactly reproduces the classical complexity classes P and NP . This provides the foundational structure needed for the subsequent analysis of how curvature, confinement, and physical resource bounds restrict realizable computation in WCT devices, and how such restrictions interact with the classical complexity landscape.

References

1. S. A. Cook. “The Complexity of Theorem-Proving Procedures.” *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing*, 1971.
2. L. A. Levin. “Universal Sequential Search Problems.” *Problems of Information Transmission*, 9(3):265–266, 1973.
3. T. Baker, J. Gill, R. Solovay. “Relativizations of the $P=?NP$ Question.” *SIAM Journal on Computing*, 4(4):431–442, 1975.
4. A. A. Razborov, S. Rudich. “Natural Proofs.” *Journal of Computer and System Sciences*, 55(1):24–35, 1997.
5. S. Aaronson, A. Wigderson. “Algebrization: A New Barrier in Complexity Theory.” *ACM Transactions on Computation Theory*, 1(1):2:1–2:54, 2009.
6. A. Lubotzky, R. Phillips, P. Sarnak. “Ramanujan Graphs.” *Combinatorica*, 8(3):261–277, 1988.
7. S. Arora, B. Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.
8. R. J. Reyes. *Structure and Derivation of Physical Constants through Wave Confinement*. Zenodo, 2025. <https://doi.org/10.5281/zenodo.15341540>.
9. R. J. Reyes. *The Geometry of Resonance: Wave Confinement Theory and the Emergence of Mass, Force, and Spacetime*. Zenodo, 2025. <https://doi.org/10.5281/zenodo.15356814>.